

Android Layouts: A Complete Guide

Reference Document

Introduction

Every screen in an Android app is built from a **layout** — a container that decides how its child views are sized and positioned. Choosing the right layout affects how easy a screen is to build, how well it performs, and how well it adapts to different screen sizes.

This document explains seven layout types: `LinearLayout`, `FrameLayout`, `RelativeLayout`, `TableLayout`, `GridLayout`, `ConstraintLayout`, and `CoordinatorLayout`. For each one it covers what it is, the attributes that control it, an example, and when to reach for it. A companion demo project that is shared puts every example below into a running app with one activity per layout.

1. LinearLayout

What it is

`LinearLayout` arranges its children in a single row or a single column — never both. It is the simplest layout and usually the first one developers learn.

How to use it

Set `android:orientation` to `horizontal` or `vertical` on the `LinearLayout` itself. Every child is then placed one after another along that axis, in the order it appears in the XML.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <TextView android:layout_width="wrap_content"
android:layout_height="wrap_content" android:text="1" />
    <TextView android:layout_width="wrap_content"
android:layout_height="wrap_content" android:text="2" />
    <TextView android:layout_width="wrap_content"
android:layout_height="wrap_content" android:text="3" />

</LinearLayout>
```

Switching `android:orientation` to `vertical` stacks the same three views top to bottom instead of left to right — nothing else about the XML needs to change.

gravity vs. layout_gravity

These two attributes are easy to confuse because they sound alike but apply in opposite directions:

`android:gravity`, set **on a container**, positions that container's *content* (its text, or its children) within itself. For example, `android:gravity="center"` on a `TextView` centers its text inside the `TextView`'s own bounds.

`android:layout_gravity`, set **on a child**, positions that child *within its parent*. For example, a child inside a vertical `LinearLayout` with `android:layout_gravity="end"` pushes itself to the right edge of the parent, while its siblings stay wherever their own `layout_gravity` places them.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="140dp"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:layout_gravity="end"
        android:text="I position myself within my parent" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:gravity="center"
        android:text="I position my own content" />

</LinearLayout>
```

layout_weight

`android:layout_weight` distributes leftover space among children proportionally, after every child's fixed size has already been accounted for. The usual pattern is to set the dimension along the layout's orientation to `0dp` for every weighted child, then let the weight values decide how the remaining space splits.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="80dp"
    android:orientation="horizontal">

    <TextView
        android:layout_width="0dp"
        android:layout_height="match_parent"
```

```

        android:layout_weight="1"
        android:text="Takes 1 share" />

<TextView
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:layout_weight="2"
    android:text="Takes 2 shares" />

</LinearLayout>

```

Here the second TextView ends up twice as wide as the first, because $1 + 2 = 3$ total shares, and it claims 2 of those 3.

When to use it

Rows and columns of views that don't overlap: toolbars, simple forms, button rows, list items. Avoid nesting several LinearLayouts inside each other for a complex screen — each nested layout costs an extra measurement pass, and deep nesting is one of the most common Android performance mistakes. For anything more complex than a simple stack, prefer ConstraintLayout.

2. FrameLayout

What it is

FrameLayout is the simplest container: it stacks its children directly on top of one another, all anchored to the top-left corner by default. Children later in the XML draw *on top of* children earlier in the XML.

How to use it

Add children directly; use `android:layout_gravity` on each child to reposition it away from the default top-left corner, and `android:layout_margin*` to offset it further.

```

<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <View
        android:layout_width="300dp"
        android:layout_height="200dp"
        android:background="@color/green" />

    <View
        android:layout_width="200dp"
        android:layout_height="300dp"
        android:layout_marginStart="30dp"

```

```
android:background="@color/pink" />
```

```
</FrameLayout>
```

The pink View is declared second, so it renders on top of the green one wherever they overlap.

When to use it

Anywhere views need to overlap: image badges, a loading spinner shown over content, a single fragment container (FrameLayout is the standard container for swapping fragments in and out), or a single child that doesn't need positioning logic. It is also the cheapest layout to measure, since it does almost no positioning work.

3. RelativeLayout

What it is

RelativeLayout positions each child relative to the parent or relative to another sibling view, using rules like “below this view” or “to the right of that view,” instead of a fixed order like LinearLayout.

How to use it

Position relative to the parent: layout_alignParentTop, layout_alignParentBottom, layout_alignParentStart, layout_alignParentEnd, layout_centerInParent, layout_centerHorizontal, layout_centerVertical.

Position relative to a sibling view: layout_toStartOf, layout_toEndOf, layout_above, layout_below. To reference a sibling this way, that sibling must have an android:id.

Align an edge with a sibling: layout_alignTop, layout_alignBottom, layout_alignStart, layout_alignEnd.

```
<RelativeLayout
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
    <TextView
```

```
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:text="@string/title" />
```

```
    <Button
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
android:layout_below="@id/title"
android:layout_toRightOf="@id/title"
android:text="@string/next" />
```

</RelativeLayout>

The button is positioned below the title *and* to its right — both rules apply at once, which is the core idea of RelativeLayout: chain enough rules together and the exact position falls out automatically.

When to use it

Simple relative positioning where LinearLayout's single-axis model doesn't fit, and the added complexity of ConstraintLayout isn't needed. In modern projects, ConstraintLayout is usually preferred for anything beyond a couple of relations, since it supports the same relative-positioning ideas plus far more (guidelines, barriers, chains, percentage sizing) without nesting.

4. TableLayout

What it is

TableLayout arranges children into rows and columns, where each TableRow is one row, and each direct child of a TableRow is one cell in that row.

How to use it

Wrap each row's cells in a TableRow. Column widths are determined automatically from the widest cell in that column across all rows — you don't set a column's width directly. Use android:stretchColumns (a 0-indexed column number) to make one column absorb any extra horizontal space.

```
<TableLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:stretchColumns="1">

    <TableRow>
        <TextView android:text="Name" />
        <TextView android:text="Score" />
    </TableRow>

    <TableRow>
        <TextView android:text="Asha" />
        <TextView android:text="92" />
    </TableRow>

</TableLayout>
```

To make a cell span multiple columns, set `android:layout_span` on that cell. To leave a column empty in one row, use `<TableRow android:layout_column="2">` style placement.

When to use it

Small, simple tabular data — a settings screen with label/value rows, a short results table. For anything larger or scrollable, a `RecyclerView` with a grid layout manager is the modern choice; `TableLayout` doesn't scroll or recycle views on its own.

5. GridLayout

What it is

`GridLayout` arranges children into a rectangular grid using row and column indexes, without needing nested containers — unlike trying to build a grid out of nested `LinearLayout` rows.

How to use it

Set `android:columnCount` and/or `android:rowCount` on the `GridLayout`. Children are placed left-to-right, top-to-bottom automatically unless you specify `android:layout_row` / `android:layout_column` explicitly. Use `android:layout_columnWeight` / `android:layout_rowWeight` to distribute extra space, similar to `LinearLayout`'s weight.

```
<GridLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="2"
    android:rowCount="3">

    <Button android:layout_columnWeight="1" android:text="1" />
    <Button android:layout_columnWeight="1" android:text="2" />
    <Button android:layout_columnWeight="1" android:text="3" />
    <Button android:layout_columnWeight="1" android:text="4" />
    <Button android:layout_columnWeight="1" android:text="5" />
    <Button android:layout_columnWeight="1" android:text="6" />

</GridLayout>
```

Six buttons with `columnCount="2"` automatically wrap into three rows of two.

When to use it

Uniform grids — keypad layouts, icon pickers, a simple calculator — where every cell is roughly the same size and you don't want the nesting cost of stacked `LinearLayout` rows. For large or scrollable grids, use a `RecyclerView` with `GridLayoutManager` instead.

6. ConstraintLayout

What it is

ConstraintLayout positions each child using constraints — rules that tie an edge of one view to an edge of another view, a guideline, a barrier, or the parent. It can express everything RelativeLayout can plus much more, with a completely flat view hierarchy (no nesting), which makes it the recommended layout for most non-trivial screens.

Basic constraints

Every view generally needs at least one horizontal and one vertical constraint or it collapses to the top-left corner:

```
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/btn"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Guidelines

A Guideline is an invisible line other views can constrain to — useful for a fixed percentage split or a fixed offset from an edge, without hardcoding numbers into every view.

```
<androidx.constraintlayout.widget.Guideline
    android:id="@+id/guideline"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    app:layout_constraintGuide_percent="0.5" />

<Button
    app:layout_constraintStart_toStartOf="@id/guideline"
    app:layout_constraintTop_toTopOf="parent" />
```

`layout_constraintGuide_percent="0.5"` places the guideline at the horizontal midpoint; `layout_constraintGuide_begin` / `layout_constraintGuide_end` place it a fixed distance from an edge instead.

Barriers

A Barrier is an invisible line that automatically tracks the edge of whichever referenced view is currently largest — useful when text length varies (e.g. localized strings) and a fixed guideline would clip.

```
<androidx.constraintlayout.widget.Barrier
    android:id="@+id/barrier"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:barrierDirection="end"
    app:constraint_referenced_ids="name,email" />

<EditText
    app:layout_constraintStart_toEndOf="@id/barrier" />
```

If “Email Address” is wider than “Name,” the barrier sits past “Email Address,” and the EditText shifts right automatically — no manual recalculation needed if the labels change.

Chains

A chain links a row (or column) of views together so they share space according to a chain style, set with `layout_constraintHorizontal_chainStyle` (or `..._Vertical_...`) on the first view in the chain:

- spread (default) — views are spaced evenly across the available room.
- packed — views are pushed together in the middle, as one group.
- spread_inside — the first and last views anchor to the parent’s edges; only the views between them spread out.

```
<Button
    android:id="@+id/b1"
    app:layout_constraintHorizontal_chainStyle="spread"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toStartOf="@id/b2" />

<Button
    android:id="@+id/b2"
    app:layout_constraintStart_toEndOf="@id/b1"
    app:layout_constraintEnd_toStartOf="@id/b3" />

<Button
    android:id="@+id/b3"
    app:layout_constraintStart_toEndOf="@id/b2"
    app:layout_constraintEnd_toEndOf="parent" />
```

Each view in the chain constrains to the next, forming an unbroken link from parent start to parent end — that unbroken chain is what the `chainStyle` attribute then arranges.

When to use it

Any screen more complex than a simple stack: forms with varying label widths, responsive layouts that need to adapt across screen sizes, or any screen currently built from several nested `LinearLayout`s. `ConstraintLayout` is Android's recommended default for building layouts today.

7. CoordinatorLayout

What it is

`CoordinatorLayout` is a “super-powered `FrameLayout`” that coordinates interactions *between* its children — most commonly, a collapsing app bar that reacts to scrolling, and a Floating Action Button (FAB) that shows a `Snackbar`. It doesn't position children on its own the way the other layouts do; instead, it reads a `layout_behavior` from each child and lets that behavior decide how the child reacts to what its siblings are doing.

How to use it

The root layout must be a `CoordinatorLayout`, typically containing an `AppBarLayout` (with a `Toolbar` or `CollapsingToolbarLayout` inside) and a scrolling content view such as `NestedScrollView` or `RecyclerView`.

```
<androidx.coordinatorlayout.widget.CoordinatorLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="200dp">

        <com.google.android.material.appbar.CollapsingToolbarLayout
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            app:layout_scrollFlags="scroll|exitUntilCollapsed">

            <androidx.appcompat.widget.Toolbar
                android:id="@+id/toolbar"
                android:layout_width="match_parent"
                android:layout_height="?attr/actionBarSize"
                app:layout_collapseMode="pin" />

            </com.google.android.material.appbar.CollapsingToolbarLayout>
        </com.google.android.material.appbar.AppBarLayout>

        <androidx.core.widget.NestedScrollView
            android:layout_width="match_parent"
```

```

        android:layout_height="match_parent"
        app:layout_behavior="@string/appbar_scrolling_view_behavior">
        <!-- scrolling content -->
    </androidx.core.widget.NestedScrollView>

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fab"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|end"
        android:layout_margin="16dp" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Floating Action Buttons

A FloatingActionButton is a circular Material button, usually for the single most common action on a screen (e.g. compose, add). Anchor it to the CoordinatorLayout with layout_gravity, and it will automatically move out of the way when a Snackbar appears.

Snackbar

A Snackbar is a brief message that slides up from the bottom, optionally with a single action:

```

Snackbar.make(view, "Action triggered", Snackbar.LENGTH_LONG)
    .setAction("Undo", v -> undoLastAction())
    .show();

```

Because it's shown inside a CoordinatorLayout, other coordinated children (like the FAB) automatically shift up while the Snackbar is visible, then settle back once it's dismissed.

Collapsing Toolbar

CollapsingToolbarLayout, nested inside an AppBarLayout, lets a tall header (often with a background image) shrink into a compact toolbar as the user scrolls the content beneath it. layout_collapseMode="parallax" on the header image makes it scroll slower than the content for a parallax effect; layout_collapseMode="pin" on the Toolbar keeps it fixed once the rest has collapsed.

When to use it

Screens with a scrolling header that should shrink, a FAB that needs to react to Snackbars or scrolling, or any custom scroll-linked interaction between two views. For a simple static toolbar with no scroll interaction, a plain Toolbar inside a LinearLayout is simpler and sufficient.

Choosing the Right Layout

Layout	Nesting	Performance	Best for
LinearLayout	Yes (deep nesting hurts performance)	Good for simple stacks	Rows/columns of views, toolbars, forms
FrameLayout	Minimal	Very fast	Overlapping views, single child, fragment containers
RelativeLayout	Low	Good	Positioning views relative to each other/parent
TableLayout	Moderate	OK for small tables	Tabular data, grid-like forms
GridLayout	Low (flat hierarchy)	Good	Uniform grids without nested weights
ConstraintLayout	Flat (no nesting needed)	Best for complex UIs	Complex, responsive screens; replaces most nesting
CoordinatorLayout	Low	Good	Coordinating scrolling behavior, FAB, Snackbar, toolbars

Companion Demo Project

The `AndroidLayoutsDemo` project puts every example in this guide into a running app: a home screen lists all seven layouts, and tapping any of them opens a dedicated activity built exactly like the XML shown above. Reading this guide alongside that project — opening the matching `.xml` file for each section — is the fastest way to see how each attribute actually behaves on screen.